

Variance Reduction for Training Neural Bayes Estimators

Kai Marns-Morris

Honours (BPhil) 2026

The University of Western Australia

Supervisors: Dr Michael Bertolacci & A/Prof Edward Cripps

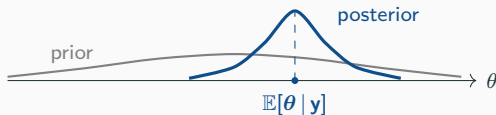
Background: neural Bayes estimators

Bayesian inference

- Fix a model: parameters θ with prior $p(\theta)$, and data $\mathbf{y}$ with likelihood $p(\mathbf{y} | \theta)$.
- Seeing $\mathbf{y}$, Bayes' rule updates the prior into the **posterior distribution**:

$$p(\theta | \mathbf{y}) \propto p(\mathbf{y} | \theta) p(\theta).$$

- The posterior is our full, updated belief about θ .

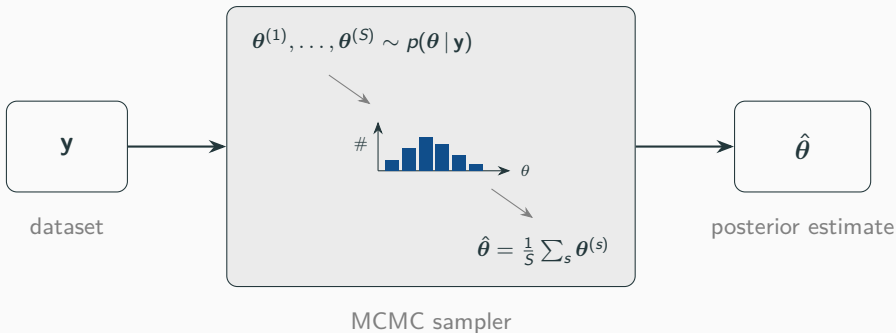


Point estimation

- Often we do not want the whole posterior — just a single **point estimate** $\hat{\theta}$ of θ .
- The usual choice is the **posterior mean** $\hat{\theta} = \mathbb{E}[\theta | \mathbf{y}]$.
- But it is **rarely available in closed form** — so we need a **procedure** that turns $\mathbf{y}$ into $\hat{\theta}$:

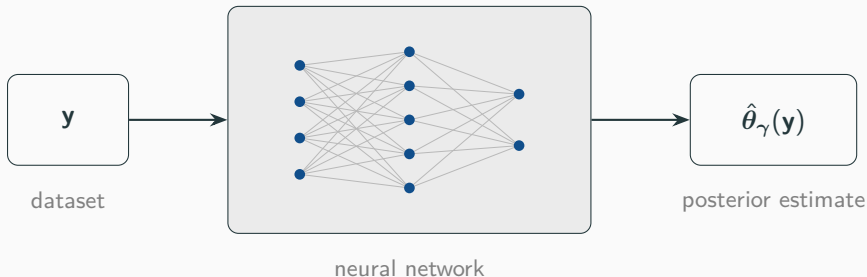


Method (1): Markov chain Monte Carlo



- Accurate and general — but must be **re-run from scratch for every new y** , and can be slow.
- Do inference for *many* datasets and you pay that cost *every time*.

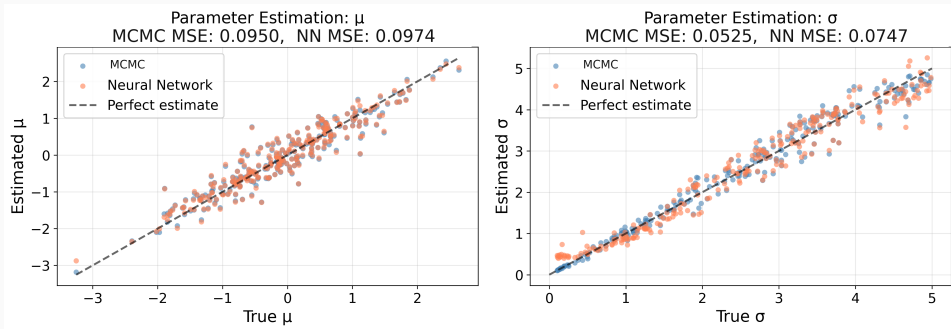
Method (2): Neural Bayes estimator



- A **neural network** $\hat{\theta}_{\gamma}(\cdot)$, trained *once* — a *neural Bayes estimator*.
- Training data is free: **simulate** pairs (θ, y) and adjust γ so $\hat{\theta}_{\gamma}(y) \approx \theta$.
- Then each new dataset is a single **forward pass** — the cost is **amortised**.

Does it work?

A simple test: $y_1, \dots, y_n \sim \mathcal{N}(\mu, \sigma^2)$ with $\mu \sim \mathcal{N}(0, 1)$ and $\sigma \sim \text{Uniform}(0, 5)$.



The network (orange) matches MCMC (blue): it has **learned the posterior-mean map**.

The loss picks which summary you estimate

The network minimises the **Bayes risk** $L(\gamma) = \mathbb{E}_{\theta, \mathbf{y}}[\ell(\theta, \hat{\theta}_\gamma(\mathbf{y}))]$ — and the loss ℓ decides *which* posterior summary it targets:

loss $\ell(\theta, \hat{\theta})$	best estimate (its minimiser)
squared $\ \theta - \hat{\theta}\ ^2$	posterior mean $\mathbb{E}[\theta \mathbf{y}]$
absolute $ \theta - \hat{\theta} $	posterior median
quantile $(\alpha - \mathbf{1}_{\{\theta < \hat{\theta}\}})(\theta - \hat{\theta})$	posterior quantile

Bayes risk is not available in closed form, so we *estimate it by Monte Carlo*:

$$L(\gamma) \approx \frac{1}{N} \sum_{i=1}^N \ell(\theta_i, \hat{\theta}_\gamma(\mathbf{y}_i)), \quad (\theta_i, \mathbf{y}_i) \sim \text{model}.$$

The idea

In training we estimate the risk by **Monte Carlo**:

$$L(\gamma) \approx \frac{1}{N} \sum_{i=1}^N \ell(\theta_i, \hat{\theta}_{\gamma}(\mathbf{y}_i)), \quad (\theta_i, \mathbf{y}_i) \sim \text{model}.$$

- So the **gradient** $g(\gamma) = \nabla_{\gamma} L(\gamma)$ that drives training is a **random quantity with variance**.
- That variance is about *how we sample the loss*, not the inference problem — **so we can reduce it**.

Idea 1: Rao–Blackwellisation

The trick

Some models have conditional conjugacy. Split the parameters $\theta = (\theta_1, \theta_2)$: conditioning on θ_2 leaves the posterior of θ_1 in **closed form**, so its mean $\mathbb{E}[\theta_1 \mid \theta_2, \mathbf{y}]$ is analytic.

- The network trains on $(\mathbf{y}, \theta)$ pairs where the target θ is a **noisy draw**. **Replace the conjugate block θ_1 with its exact mean $\mathbb{E}[\theta_1 \mid \theta_2, \mathbf{y}]$ — a cleaner label.**
- This is classical **Rao–Blackwell**: replacing a sampled piece by its conditional mean can only *reduce* variance.

Example — Bayesian linear regression: given the noise variance $\sigma^2 (= \theta_2)$, the coefficients $\beta (= \theta_1)$ are Gaussian, so $\mathbb{E}[\beta \mid \sigma^2, \mathbf{y}]$ is closed-form.

Bayesian linear regression: a cleaner target

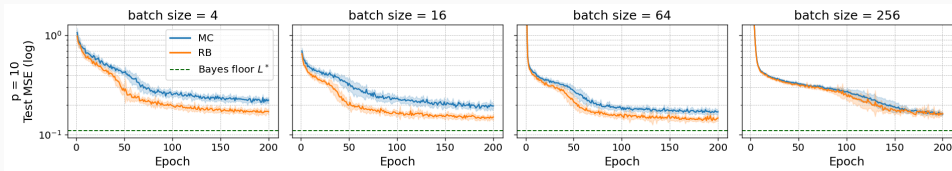
Model $\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \sigma^2 I)$ — the parameters are the coefficients $\boldsymbol{\beta}$ and the noise variance σ^2 .

- **Conjugacy:** given σ^2 , the posterior of $\boldsymbol{\beta}$ is Gaussian, so its mean $\mathbb{E}[\boldsymbol{\beta} | \sigma^2, \mathbf{y}]$ is a **closed-form formula**.
- Train on that exact mean instead of a sampled $\boldsymbol{\beta}$. Under squared error the loss splits (given $\sigma^2, \mathbf{y}$):

$$\mathbb{E}[\|\boldsymbol{\beta} - \hat{\boldsymbol{\beta}}\|^2 | \sigma^2, \mathbf{y}] = \|\hat{\boldsymbol{\beta}} - \mathbb{E}[\boldsymbol{\beta} | \sigma^2, \mathbf{y}]\|^2 + \underbrace{\text{Tr}(\text{Var}(\boldsymbol{\beta} | \sigma^2, \mathbf{y}))}_{\text{constant, drops from } \gamma}.$$

The network regresses on the exact conditional mean $\mathbb{E}[\boldsymbol{\beta} | \sigma^2, \mathbf{y}]$ — same target, **less noise**.

Does it pay off?



Bayesian linear regression, $p = 10$ coefficients; mean ± 1 s.d. over repeated training runs.

- The Gibbs sampler gives the Bayes-optimal floor L^* (green).
- RB reaches a **lower test error** at the small and moderate batches — where the training gradient is noisiest.
- The advantage **shrinks as the batch grows**: by batch 256 the curves meet.

In general

Nothing used linear regression in particular. Split any parameters $\theta = (\theta_1, \theta_2)$ with θ_1 **conditionally conjugate**, and replace its sampled target by $\mathbb{E}[\theta_1 | \theta_2, \mathbf{y}]$.

- The Rao–Blackwellised gradient is the standard one **conditioned** on $(\theta_2, \mathbf{y})$, so the law of total variance gives

$$\text{Var}(g) - \text{Var}(g_{\text{RB}}) = \mathbb{E}[\text{Var}(g | \theta_2, \mathbf{y})] \succeq 0.$$

- **Never noisier — for any model and any architecture.** For a linear network, the guarantee even reaches the fitted weights.

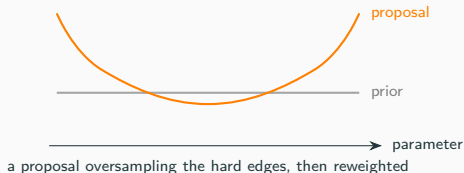
Largest gain early in training and on weakly-identified parameters — when the optimiser most needs a clean signal.

Idea 2: importance sampling

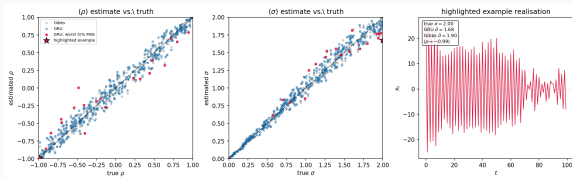
A more general lever

No conjugacy required — unlike Rao–Blackwell.

- Simulate from a **proposal** that oversamples the informative regions, then **reweight** back to the original risk (unbiased for any proposal).
- But **no guarantee**: bad weights can *increase* variance and shrink the effective sample size.

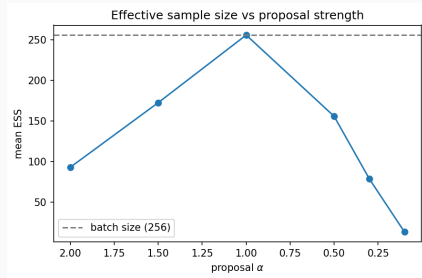


Importance sampling: the result



On an AR(1) model, the GRU already tracks the Gibbs (Bayes) reference across the prior — **little error left to recover.**

Why it fails here: the estimator already sits near the Bayes floor, and the little error that remains lives at the $|\rho| \rightarrow 1$ boundary — which the proposals do not target.



Push the proposal hard and the effective sample size **collapses** — only adding noise.

Conclusion

Training a neural Bayes estimator is itself Monte Carlo — so its training variance can be reduced.

- **Rao–Blackwellisation** (needs a conjugate block): a *provably* lower-variance training gradient, for any model or architecture — on linear regression it trains to a lower error, most where the batch is small.
- **Importance sampling** (needs nothing special): more general, but unguaranteed and problem-dependent — here it did not help.

Future work: many more variance-reduction techniques remain to be tried.

Thank you